

rdma_vip

RDMA VIP设计与TLM编程框架

根据您提供的技术资料，以下是对RDMA requester（请求端）接收侧错误处理、数据写入及状态管理核心机制的整合摘要。

一、错误处理与重传机制

该机制是保障可靠传输（如RC服务）的核心，由请求端主导。

1. PSN检查与丢包检测：

- 请求端和响应端均通过**包序列号（PSN）** 检查乱序或丢包。
- 若请求端收到非期望PSN的响应包，或收到响应端的**否定确认（NAK）**，则判定丢包/错误。

2. 触发重传：

- 当请求端未收到预期的ACK或响应（如RDMA Read Response）时，会触发重传。
- 关键特性：**对于RDMA READ操作，重传请求无需与原始请求完全一致，可**仅重传未成功响应的部分数据**，其PSN需位于“重复PSN区域”。

3. 内存访问错误：

- 由响应端在收到请求时，通过验证 **R_Key** 和地址范围进行权限检查。
- 若校验失败（如无效R_Key、越界），响应端会返回携带错误码的NAK通知请求端。

4. 实现要点：

- 重传管理：**基于PSN状态跟踪，而非维护独立数据包队列。
- 次数与退避：**规范未明确限定重传次数和指数退避策略，持续失败可能导致QP进入错误状态。
- 协议差异：**iWARP（基于TCP）支持选择性重传，开销低；RoCEv2（基于UDP）采用Go-Back-N重传，开销较高。

二、接收Read Response与写入本地内存流程

请求端收到Read Response后，其HCA硬件自动执行以下步骤，将数据写入本地内存：

1. 解析与校验：

- 解析报文头，提取 **目标STag（即L_Key）** 和 **Tagged Offset**。

- 执行严格的**PSN顺序校验**，确保可靠传输。
- 2. **地址转换与权限检查：**
 - 使用**L_Key**查找本地**内存区域（MR）** 上下文。
 - **验证权限**：确认MR具备**local_write**权限。
 - **地址转换**：结合Tagged Offset和MR信息，计算出最终的**物理地址（PA）**。
 - **范围检查**：确保操作地址范围在MR内。
- 3. **执行DMA写入：**
 - 校验通过后，HCA的DMA引擎将Payload**直接写入**计算得到的物理地址，过程无需CPU参与。
- 4. **完成通知：**
 - **关键时序**：仅当**最后一个**Read Response包成功写入后，硬件才生成**完成队列条目（CQE）**。
 - 应用**必须轮询到CQE后**，才能安全读取Buffer中的数据。

三、SQ WQE完成状态与QP状态机管理

请求端通过CQE和QP状态机来管理WQE的完成状态。

1. **CQE生成触发条件：**
 - **SEND**：在可靠连接中，收到对端ACK后生成。
 - **RDMA WRITE**：收到对端ACK后生成。
 - **RDMA READ**：收到所有响应数据并写入本地内存后生成。
 - **ATOMIC**：收到正确的操作响应（AETH）后生成。
 - **保序规则**：同一QP内，CQE的产生顺序与WQE下发顺序一致。
2. **QP状态机关键状态：**
 - **RST**：初始状态，QP不可用。
 - **INIT**：可接收但不处理消息。
 - **RTR**：接收队列（RQ）可正常工作。
 - **RTS**：**关键状态**，发送队列（SQ）可正常工作，用户WR在此状态下才会被处理并最终产生CQE。
 - **ERR/SQEr**：错误状态，停止处理WQE。

3. 错误处理流程：

- **错误类型**：分为立即错误（WR下发时）、完成错误（硬件执行失败，通过CQE报告）、异步错误（QP/链路级）。
- **恢复方式**：当QP因错误进入ERR或SQEr状态后，**必须由上层通过Modify QP将其重置回RST状态**，然后重新初始化（RST→INIT→RTR→RTS）才能恢复通信。

总结

RDMA请求端的接收侧逻辑是一个由硬件保障的精密流程。其核心在于：通过**PSN与ACK/NAK机制**实现可靠传输与重传；通过**Key（R_Key/L_Key）机制**实现安全的地址转换与内存保护；通过**CQE和QP状态机**为上层提供明确、保序的完成语义与错误恢复路径。整个过程旨在实现高带宽、低延迟且可靠的数据传输。

二、协议层抽象与TLM事务定义

协议层核心构建块抽象

RDMA协议层的行为由一系列核心的、标准化的元素构成，这些元素是TLM建模的直接对象。

- WQE/事务请求抽象**：WQE（Work Queue Element）是软件下发给硬件的一份“任务说明”，描述了硬件需要执行的操作及详细信息。它是TLM模型中“事务请求”的源头。根据协议规范，一个WQE必须包含以下关键信息以定义一个完整的操作：
 - **操作码 (opcode)**：指定操作类型，如SEND、RDMA WRITE、RDMA READ、ATOMIC。
 - **本端数据缓冲区信息**：包括数据起始虚拟地址 (local_va)、长度 (length) 以及用于验证权限和地址转换的本地键 (l_key)。
 - **对端信息（针对RDMA操作）**：包括远端内存的起始虚拟地址 (remote_va) 以及对应的远程访问密钥 (r_key)。
 - **其他控制信息**：如可选立即数 (immediate_data)、围栏标志 (fence) 等属性。
- 协议报文头部抽象**：WQE中的信息最终被硬件转化为网络报文。因此，TLM事务必须能精确反映各类协议头部所承载的语义。
 - **RETH (RDMA Extended Transport Header)**：是RDMA WRITE和READ操作请求包的强制性头部，定义了操作的目标内存地址、权限和大小。其包含三个核心字段：
 - **虚拟地址 (VA)**：64位，远端内存的起始虚拟地址。
 - **远程密钥 (R_Key)**：32位，用于授权和验证对远端指定内存区域的访问权限。
 - **DMA长度 (DMALen)**：32位，此次RDMA操作要传输的数据总长度。

- **原子操作头部 (AtomicETH)**: 用于ATOMIC操作, 包含目标地址、R_Key以及具体的操作数 (如Swap Data, Compare Data)。
 - **确认与完成头部 (AETH)**: 用于携带ACK确认或错误状态 (syndrome), 是判断事务完成状态 (成功/失败) 的关键。
3. **交互流程与状态抽象**: RDMA操作在Requester (请求端) 和Responder (响应端) 之间的交互, 构成了事务的生命周期。TLM模型需要为此生命周期的关键阶段建模:
- **建链 (握手) 与QP状态**: QP (Queue Pair) 需经历 RESET → INIT → RTR → RTS 的状态迁移才能进行数据传输。建链过程交换的初始PSN、QPN等参数是后续事务正确执行的基础。
 - **数据传输流程**:
 - **RDMA WRITE**: 请求端发送含RETH和Payload的包, 响应端直接写入内存后回复ACK。
 - **RDMA READ**: 请求端发送Read Request (含RETH), 响应端返回Read Response (含AETH和Payload)。这是一个明确的“请求-响应”事务对。
 - **ATOMIC**: 请求端发送Atomic Request (含AtomicETH), 响应端执行原子操作后返回Atomic Acknowledge (含AETH和AtomicAckETH)。
 - **完成与错误抽象**: 事务的最终状态通过CQE (Completion Queue Element) 体现。CQE中必须包含状态 (status)、操作码 (opcode)、关联的QP号 (qp_num)、软件请求ID (wr_id) 以及可能的错误码和立即数。

TLM事务类定义

在SV/UVM验证环境中, 上述协议层抽象需要被实例化为可编程的类 (Class), 以进行事务级的建模、传输和检查。

1. 事务类型枚举 (rdma_opcode_t)

基于InfiniBand规范 (如Table 38 OpCode字段), 定义一个枚举类型来清晰标识所有支持的RDMA操作。这是事务类的首要标识。

Plain Text

```
1 typedef enum bit [7:0] {
2     // 可靠连接 (RC) 服务下的操作示例
3     RC_SEND_FIRST           = 8'b0010_0000,
4     RC_SEND_MIDDLE         = 8'b0010_0001,
5     RC_SEND_LAST           = 8'b0010_0010,
6     RC_SEND_ONLY           = 8'b0010_0100,
7     RC_SEND_ONLY_WITH_IMM  = 8'b0010_0101,
```

```

8
9     RC_RDMA_WRITE_FIRST      = 8'b0011_0000,
10    RC_RDMA_WRITE_MIDDLE     = 8'b0011_0001,
11    RC_RDMA_WRITE_LAST       = 8'b0011_0010,
12    RC_RDMA_WRITE_ONLY       = 8'b0011_0011,
13    RC_RDMA_WRITE_ONLY_WITH_IMM = 8'b0011_0101,
14
15    RC_RDMA_READ_REQUEST      = 8'b0011_0100,
16
17    RC_ATOMIC_CMP_SWP         = 8'b0011_1000,
18    RC_ATOMIC_FETCH_ADD       = 8'b0011_1001,
19
20    // 确认与响应
21    RC_ACK                     = 8'b1000_0001,
22    // ... 可根据需要添加UC, UD等服务的操作码
23 } rdma_opcode_t;

```

2. 分层头部数据结构

定义对应的类来表示各层协议头部，这些类将作为事务的属性。

Plain Text

```

1 // RDMA扩展传输头部 (RETH)
2 class rdma_reth extends uvm_sequence_item;
3     rand bit [63:0] virtual_addr;
4     rand bit [31:0] r_key;
5     rand bit [31:0] dma_length;
6     // ... pack/unpack, constraints 等方法
7 endclass
8
9 // 原子操作扩展头部 (AtomicETH)
10 class rdma_atomic_eth extends uvm_sequence_item;
11     rand bit [63:0] virtual_addr;
12     rand bit [31:0] r_key;
13     rand bit [63:0] swap_data;
14     rand bit [63:0] compare_data; // 用于CmpSwap
15     // ... 方法
16 endclass
17
18 // 确认扩展头部 (AETH)
19 class rdma_aeth extends uvm_sequence_item;
20     rand bit [7:0] syndrome; // 包含确认状态和错误码
21     // ... 方法
22 endclass

```

3. 核心RDMA事务类 (rdma_transaction)

此类是TLM建模的中心，它将一次完整的RDMA操作抽象为一个事务对象。

Plain Text

```
1 class rdma_transaction extends uvm_sequence_item;
2     `uvm_object_utils(rdma_transaction)
3
4     // 事务标识与类型
5     rand rdma_opcode_t      opcode;
6     rand int                local_qpn;
7     rand int                remote_qpn;
8     rand bit [23:0]        start_psn; // 起始包序列号
9
10    // 内存访问语义（核心字段）
11    rand bit [63:0]         local_vaddr; // 本地虚拟地址（源或目标）
12    rand bit [63:0]         remote_vaddr; // 远端虚拟地址
13    rand bit [31:0]         l_key;        // 本地内存键
14    rand bit [31:0]         r_key;        // 远端内存键
15    rand int unsigned       length;       // 操作数据总长度
16    rand bit [31:0]         immediate_data; // 可选立即数
17
18    // 数据负载（用于建模实际传输的数据）
19    rand byte               data_payload[];
20
21    // 协议头部实例（根据操作码动态使用）
22    rdma_reth               reth;
23    rdma_atomic_eth         atomic_eth;
24    rdma_aeth               aeth;
25
26    // 事务状态与生命周期
27    typedef enum {TRAN_PENDING, TRAN_REQ_SENT, TRAN_RESP_RECV,
TRAN_COMPLETED, TRAN_ERROR} trans_state_e;
28    trans_state_e           state = TRAN_PENDING;
29    bit                    completed;
30    bit                    error;
31    bit [31:0]             completion_status; // 映射到CQE状态
32
33    // 约束：确保事务参数与操作类型语义一致
34    constraint valid_semantic_c {
35        // RDMA WRITE/READ操作必须有有效的远端地址和R_Key
36        (opcode inside {RC_RDMA_WRITE_FIRST, RC_RDMA_WRITE_ONLY,
RC_RDMA_READ_REQUEST}) ->
37            (remote_vaddr != 0 && r_key != 0 && length > 0);
38        // ATOMIC操作需要AtomicETH
39        (opcode inside {RC_ATOMIC_CMP_SWP, RC_ATOMIC_FETCH_ADD}) ->
```

```

40         (atomic_eth != null);
41     // SEND操作可能携带立即数,但不使用RETH
42     (opcode inside {RC_SEND_ONLY_WITH_IMM}) -> (immediate_data != 0);
43     // 长度约束 (如IB规范定义的0到2^31字节)
44     length <= (1 << 31);
45 }
46
47 // 构造函数与基础方法
48 function new(string name = "rdma_transaction");
49     super.new(name);
50 endfunction
51
52 // 方法: 根据事务属性, 构建对应的协议头部
53 virtual function void build_headers();
54     case(opcode)
55         RC_RDMA_WRITE_FIRST, RC_RDMA_WRITE_ONLY, RC_RDMA_READ_REQUEST:
begin
56         reth = new();
57         reth.virtual_addr = this.remote_vaddr;
58         reth.r_key = this.r_key;
59         reth.dma_length = this.length;
60     end
61     RC_ATOMIC_CMP_SWP, RC_ATOMIC_FETCH_ADD: begin
62         atomic_eth = new();
63         atomic_eth.virtual_addr = this.remote_vaddr;
64         atomic_eth.r_key = this.r_key;
65         // ... 设置操作数
66     end
67 endcase
68 endfunction
69
70 // 方法: 处理响应, 更新事务状态 (例如, 根据收到的AETH)
71 virtual function bit process_response(rdma_aeth resp_aeth);
72     this.aeth = resp_aeth;
73     if (resp_aeth.syndrome == 0) begin
74         this.state = TRAN_COMPLETED;
75         this.completed = 1;
76         this.error = 0;
77     end else begin
78         this.state = TRAN_ERROR;
79         this.completed = 1;
80         this.error = 1;
81         this.completion_status = resp_aeth.syndrome;
82     end
83     return this.completed;
84 endfunction
85

```

```
86 // ... 其他方法，如：pack()/unpack()用于与底层BFM交互，compare()用于结果比对等。  
87 endclass
```

此 `rdma_transaction` 类构成了VIP中TLM通信的通用“货币”，后续的Agent（如`requester_tx_agent`）将负责生成和发送此类事务，而`responder_rx_agent`和`requester_rx_agent`则负责接收、解析并响应相应的事务。

基于已提供的文档内容，以下是关于RDMA Requester接收端错误处理、重传机制及操作流程的综合回答。所有信息均严格源自文档，无任何外部补充。

一、RDMA Requester接收端错误处理机制

核心机制概述：

RDMA请求端（Requester）的错误处理高度依赖于**服务类型**。可靠服务（如RC）通过PSN、ACK/NAK和重传确保可靠性；不可靠服务（如UD）则无重传保证。

1. PSN乱序与数据包丢失检测

- **可靠服务（RC）：**
 - 请求端和响应端均通过**包序列号（PSN）**检测乱序或丢失的数据包。
 - 若请求端未按序接收响应包（如收到PSN=3但未收到PSN=2），会触发错误恢复流程。
- **不可靠服务（UD）：**
 - 不支持ACK机制，请求端无法直接感知丢包或乱序。
 - 响应端可能检测到序列错误并进行本地记录，但**不通知请求端**。

2. 超时重传机制

- **可靠服务（RC）：**
 - 请求端在**未收到响应（如ACK或RDMA Read Response）或收到NAK**时触发重传。
 - **RDMA Read重传特性：**重试的请求**无需与原始请求完全一致**，可以仅重传未成功响应的部分数据。重传请求的PSN需位于**重复PSN区域**。
 - 重传过程对上层应用透明，除非目标不可达或所有路径失效。
- **不可靠服务（UD）：**
 - 不支持ACK或重传。出错或丢失的报文被直接静默丢弃。

3. 内存访问错误处理

- **R_Key验证**：请求端在发起操作（如RDMA Write/Read）前，远端（Responder）会验证R_Key的有效性和访问权限。若无效（如保护错误），响应端会返回 **NAK（否定确认）**。
- **错误上报**：
 - 对于ATOMIC操作，若执行失败（如R_Key保护错误），响应端通过NAK通知请求端。
 - 若操作执行后但响应丢失（如硬件致命错误），请求端无法确知状态，需由上层协议处理。
- **资源管理**：请求端需确保未完成的ATOMIC和RDMA Read操作不超过响应端协商的资源上限，否则可能引发错误。

4. 错误恢复流程摘要

- **可靠服务（RC）**：请求端通过重传、备用路径等方式**透明恢复**错误，通常无需应用介入。仅当目标不可操作或所有路径失效时，QP处理才暂停。
- **不可靠服务（UD）**：错误包可能被丢弃或不交付，无重传机制。

5. 协议差异（iWARP vs RoCE v2）

- **iWARP**：基于TCP，支持**选择性重传**（仅重传丢失包），开销较低。
- **RoCE v2**：基于UDP，采用**Go-back-N重传**（从丢包处重传所有后续包），开销较大，依赖于无损网络环境。

二、RDMA Requester重传机制实现细节

核心机制：

重传是由请求端硬件（HCA）主导的、基于PSN状态和ACK/NAK反馈的快速恢复机制。

1. 重传触发与定时器管理

- 触发条件主要基于 **ACK超时未到达**或接收到 **NAK**。
- 规范未规定标准化定时器实现，通常由HCA厂商具体实现。

2. 重传队列维护

- RDMA的重传是**基于PSN状态**管理的，而非独立数据包队列。HCA跟踪已发送但未确认的PSN范围。
- **RDMA READ操作的重传**允许**部分重传**，请求端可精细记录哪些数据段已确认，仅重传未成功部分。

3. 重传次数限制

- 规范文档未明确全局重传次数上限。

- 持续的失败重传可能引发更高级别错误（如RNR超时），导致QP进入错误状态并上报异步事件。

4. 指数退避策略

- 文档内容**没有直接提及**在RDMA层面使用类似TCP的指数退避策略。
- 为实现低延迟，RDMA重传更倾向于由硬件快速处理，超时时间可能与网络RTT相关并由厂商优化设定。

三、RDMA Requester处理Read Response的具体流程

整体流程： 请求端网卡（HCA）接收Read Response报文，校验后通过DMA将数据写入本地应用内存。

1. 数据包解析

- 解析报文头部（如IB的BTH，iWARP的DDP/RDMAP头）。
- 提取关键信息：**操作码（OpCode）** 标识为Read Response，以及**S Tag（R_Key）** 和 **Tagged Offset（TO）**。

2. PSN检查（可靠服务必需）

- 请求端维护期望接收的PSN (rcv_psn)。
- 检查报文BTH中的PSN：
 - **等于期望值**：接受处理，更新期望PSN。
 - **小于期望值**：为重复包，丢弃其Payload（可能仍需发ACK）。
 - **大于期望值**：表明有包丢失，暂缓处理并可能触发重传（如发送NAK）。

3. 内存权限与地址转换

- 使用提取的 **R_Key** 查找本地**Memory Region (MR)** 的转换与保护表。
- 验证权限（需包含local_write）并检查操作范围是否合法。
- 结合 **TO** 和 MR 基地址，计算目标**虚拟地址(VA)**，并通过页表转换为**物理地址(PA)**。

4. DMA写入本地内存

- 所有校验通过后，HCA的**DMA引擎**将报文Payload直接写入计算得到的**物理内存地址**。
- 此过程为零拷贝，CPU不参与。

5. 完成通知与顺序保证

- **CQE生成**：当最后一个Read Response被成功接收并写入内存后，请求端硬件生成**完成队列条目（CQE）**放入CQ。
- **关键时序规则**：
 - 应用程序**必须等待收到此CQE后**，才能安全读取目标Buffer中的数据。
 - 若之前的WR设置了 **Fence（栅栏）** 属性，则该Read Request会被阻塞，直到它之前所有已发布的RDMA Read和Atomic操作完成，以保证跨操作顺序。

四、RDMA Requester的SQ WQE完成状态管理

1. CQE生成的触发条件

- **可靠连接服务（RC）**：CQE通常在接收到对端的**ACK**后生成。
 - **SEND操作**：收到对端ACK后生成CQE。
 - **RDMA WRITE操作**：收到对端ACK后生成CQE。
 - **RDMA READ操作**：收到并处理完最后一个Read Response后生成CQE。
 - **ATOMIC操作**：收到正确的ATOMIC响应（含AETH）即视为确认，生成CQE。
- **不可靠服务（UD）**：无ACK，CQE可能在数据包发送后立即生成。
- **顺序保证**：同一工作队列（WQ）内的WQE，其CQE必须按照WQE下发（posted）的顺序产生。

2. QP完成状态机（关键状态）

QP必须处于**RTS（Ready to Send）**状态，其SQ才能正常工作并最终产生CQE。正常状态转换由用户通过`ibv_modify_qp()`驱动：

RST → INIT → RTR → RTS

- **ERR（错误状态）**：发生严重错误时进入，QP停止处理所有WQE。
- **SQEr（发送队列错误状态）**：当某个Send WR发生**完成错误**（通过错误CQE报告）时进入。

3. 错误状态处理流程

- **错误检测**：
 - **完成错误**：硬件执行WQE时发生（如内存访问违例、超时），通过状态为**FAILED**的CQE上报。
 - **异步错误**：与特定WQE无关的QP或链路错误（如链接断开），通过异步事件通道上报。
- **错误恢复**：

- a. 应用通过轮询CQ (`ibv_poll_cq`) 或事件通道获知错误。
 - b. QP进入**SQEr**或**ERR**状态后，**无法继续正常通信**。
 - c. 必须由上层应用通过**`ibv_modify_qp()` 将QP状态**先修改回RST**。
 - d. 然后重新执行完整的状态机初始化流程 (RST → INIT → RTR → RTS) 以恢复通信。
- 根据提供的文档内容，以下是关于**RDMA Requester接收端错误处理机制及相关流程**的系统性总结与分析。文档内容主要基于InfiniBand (IB) 规范，部分参考iWARP/RoCE协议，完整勾勒了Requester在处理响应、应对错误和完成工作请求的核心机制。
-

一、RDMA Requester接收端的错误处理机制概览

RDMA的可靠性与错误处理高度依赖服务类型 (Reliable vs. Unreliable)，核心基于**包序列号 (PSN)** 和**确认/否定确认 (ACK/NAK)** 机制。

1. PSN乱序与数据包丢失检测

- **可靠服务 (RC、RD)**：Requester和Responder都维护PSN，用于检测乱序或丢失的包。若Requester未按序收到响应（如收到PSN=3但未收到PSN=2），会触发错误处理流程。
- **不可靠服务 (UC、UD)**：不支持ACK，Requester无法直接感知丢包或乱序。Responder可能检测到序列错误但不通知Requester。

2. 超时重传机制

- **可靠服务**：Requester未收到预期响应（如ACK或RDMA Read Response）时，会触发重传。重传对应用透明，除非所有路径失败。
- **以RDMA Read为例**：如果Request未收到Response，Requester会重试。**重试的Read Request可以只读取未成功响应的部分数据，且其PSN必须位于“重复PSN区域”内**，以便Responder识别。
- **不可靠服务**：不支持ACK或重传，丢失的包被静默丢弃。

3. 内存访问错误的检测与处理

- **R_Key验证**：Requester在发起操作时，Responder会验证其提供的R_Key（远程内存访问密钥）和权限。若无效或权限不足，Responder会返回NAK（如保护错误）。
- **错误上报**：例如，ATOMIC操作失败（如R_Key错误），Responder通过NAK通知Requester。若操作执行后响应丢失，需由上层协议处理。
- **资源管理**：Requester需确保并发操作数不超过协商上限，否则可能引发错误。

协议差异：iWARP基于TCP，支持选择性重传，开销低；RoCEv2基于UDP，采用Go-back-N重传，开销高，依赖无损网络。

二、RDMA Read Response处理流程详解

当Requester（即Read操作发起端）收到Read Response后，网卡（HCA/RNIC）执行以下硬件操作：

1. 数据包解析

- 解析协议头（IB的BTH或iWARP的RDMAP/DDP头），获取 **OpCode**（确认是Read Response）和关键信息：**STag**（或R_Key）与 **Tagged Offset**，这些直接来自原始Read Request。

2. PSN检查（关键可靠性保证）

- Requester维护期望接收的PSN（rcv_psn）。
- 检查规则：
 - **PSN == 期望值**：接受，处理后续，更新期望PSN。
 - **PSN < 期望值**：是重复包（可能ACK丢失）。**丢弃其Payload**，但可能仍需发送ACK。
 - **PSN > 期望值**：有包丢失。**暂缓处理此包及后续包**，可能触发NAK重传。

3. 内存权限与地址验证

- 使用解析出的 **R_Key** 查找本地的**内存区域上下文**。
- 执行三项检查：
 - i. **Key匹配**：检查R_Key值是否有效。
 - ii. **权限检查**：确认该MR的权限包含local_write（本地写权限）。
 - iii. **范围检查**：确认目标地址和操作长度在MR的合法虚拟地址范围内。

4. 地址转换与数据写入本地内存

- **地址转换**：结合Tagged Offset和MR的基地址信息，通过查找页表，将**虚拟地址（VA）**转换为**物理地址（PA）**。
- **DMA写入**：HCA的DMA引擎，将Response报文中的**Payload**直接写入计算得到的**PA**。此过程**零拷贝**，无需CPU参与。如果数据跨多个包，则根据TO偏移连续写入。

5. 完成通知与顺序保证

- **CQE产生**：当**最后一个**Read Response被成功接收并写入内存后，Requester硬件生成**CQE**放入CQ。
- **关键时序要求**：应用**必须等到CQE后**，才能安全读取目标Buffer。

- **Fence操作遵守**：如果该Read Request被标记了Fence属性，则必须等待它之前所有已发布的RDMA Read和Atomic操作完成后，才能执行。

三、SQ WQE完成状态与错误处理流程

Requester通过CQE和QP状态机管理SQ WQE的完成。

1. CQE生成的触发条件（因操作类型而异）

- **可靠操作（SEND, WRITE, READ, ATOMIC）**：通常在收到对端的确认（ACK或特定响应）后生成CQE。
- **不可靠操作**：可能在数据包成功发送/接收后（无ACK流程）生成。
- **带立即数（Immediate Data）的操作**：立即数本身会触发一个CQE。
- **保序规则**：同一WQ内的CQE严格按照WQE下发顺序产生。

2. QP完成状态机

QP必须处于**就绪发送（RTS）** 状态，其SQ才能正常工作并最终产生CQE。关键状态转换路径为：

RST（复位） → INIT → RTR（可接收） → RTS（可发送）。

该转换由应用通过`ibv_modify_qp()`主动驱动。

3. 错误状态处理流程

- **错误分类**：
 - **立即错误**：`ibv_post_send`时由驱动发现（如参数非法），直接返回错误。
 - **完成错误**：硬件执行WQE时发生（如内存保护错误、超时），通过 **CQE（状态失败）** 上报。
 - **异步错误**：与特定WQE无关的QP或链路级错误（如断开），通过异步事件通道上报。
- **错误恢复**：
 - i. 出错后，QP可能自动进入 **SQEr（发送队列错误）** 或 **ERR（错误）** 状态，并停止处理所有WQE。
 - ii. 应用通过轮询CQ或异步事件感知错误。
 - iii. 应用必须调用`ibv_modify_qp()`，将QP**状态重置回RST**，然后按**RST->INIT->RTR->RTS**流程重新初始化，才能恢复通信。

总结：RDMA Requester的接收端错误处理是一个由硬件主导、协议定义的严密过程。通过**PSN**和**ACK/NAK**实现可靠性，通过**R_Key**和**状态机**保障安全性与可控性，最终通过**CQE**向软件提供明

确的完成语义。整个机制在追求高性能的同时，确保了数据传输的可靠与准确。

五、requester_rx_agent实现框架

requester_rx_agent是RDMA VIP中负责模拟请求端（Requester）接收侧行为的关键组件。它在可靠连接（RC）服务下，专门处理对端Responder回复的Read Response数据包，完成将远端数据写入本地内存的最终操作，并产生完成通知。其设计核心在于精确模拟硬件HCA接收侧的协议行为、状态管理和错误处理流程。

5.1 功能概述与定位

该Agent在VIP中的数据流角色非常明确：它是Requester端数据通路的“终点”。

当requester_tx_agent发出一个RDMA Read Request后，本Agent等待并处理返回的响应流，实现完整的“请求-响应”单边操作闭环。

其核心职责包括：

1. **协议包解析**：从网络接口接收并解析Read Response报文，提取BTH、AETH及Payload。
2. **顺序与可靠性保障**：执行严格的PSN（Packet Sequence Number）顺序检查，处理乱序、丢包场景，维护重传状态。
3. **本地内存操作**：进行地址转换（VA -> PA）和内存保护检查（L_Key验证），触发DMA写入模型将数据存入本地内存缓冲区。
4. **完成事件生成**：在最后一个Response包处理后，生成对应的CQE（Completion Queue Element），通知上层应用请求完成。
5. **错误注入与处理**：模拟各类传输错误和本地错误，验证系统的容错能力。

5.2 基于TLM的事务处理流程框架

Agent内部遵循一个清晰的、可分阶段配置的事务处理管道（Pipeline）。下图概述了其核心数据处理流程：

```
<code>graph TD
    A[接收Read Response报文] --> B[报文解析器<br/>提取BTH/AETH/Payload];
    B --> C[PSN管理器<br/>检查序列号];
    C -- PSN匹配 --> D[MR查找与权限检查<br/>使用L_Key查找MR, 计算PA];
    C -- PSN超前(丢包) --> E[触发重传逻辑/暂停处理];
    C -- PSN重复 --> F[丢弃Payload, 可发送ACK];
    D --> G[DMA写入引擎<br/>将Payload写入本地内存模型];
    G --> H[完成判断<br/>是否为最后一个Response?];
    H -- 是 --> I[CQE生成器<br/>生成CQE, 推送至CQ];
    H -- 否 --> J[更新期望PSN, 等待下一包];
    I --> K[通知Reference Model<br/>更新事务状态为COMPLETED];
    subgraph "错误处理路径"
        E --> L[权限/地址错误] --> M[丢弃Payload, 生成错误CQE];
    end
    D -- 检查失败 --> L;
</code>
```

5.2.1 输入接口与事务对象

Agent通过TLM接口接收两种输入：

1. **网络数据报文**：来自物理层或链路层BFM的 `rdma_packet` 事务，其格式需包含完整的协议头部（LRH, BTH, 可能的AETH）和Payload。
2. **本地上下文信息**：来自上层或配置数据库的 `rdma_qp_context` 事务，包含当前QP的状态（必须为RTS）、SQ WQE的原始信息（特别是 `wr_id`, `local_va`, `l_key` 等），用于关联响应包与原始请求。

5.2.2 核心处理阶段详解

阶段一：报文解析与校验

- **解析器**：根据BTH中的OpCode识别是否为RDMA_READ_RESPONSE_ONLY/FIRST/MIDDLE/LAST。提取关键字段：
 - `BTH.PSN`：用于顺序检查。
 - `AETH.syndrome`（若存在）：来自Responder的ACK/NAK状态码。
 - `Payload`：待写入的原始数据。
- **初始过滤**：若QP不在RTS状态，直接丢弃报文。

阶段二：PSN管理与顺序保障

- **PSN管理器**维护一个 `expected_psn` 寄存器，该值来源于初始建链协商及已成功处理的报文序列。
- **检查逻辑**：
 - `PSN == expected_psn`：正常接收，进入下一阶段，`expected_psn++`。
 - `PSN < expected_psn`：重复包。**行为可配置**：可丢弃Payload，但可能仍需模拟发送ACK（用于协议合规性测试）。
 - `PSN > expected_psn`：检测到丢包。触发**重传等待逻辑**。可配置为暂停处理后续包，并启动重传定时器模型，或直接断言错误以测试超时恢复。

阶段三：地址转换与权限检查（关键安全步骤）

- **MR查找**：利用从关联的SQ WQE中获取的 `l_key`，查询**内存区域（MR）转换表**（在VIP中通常由 `reg_model` 维护），获取MR的基地址、长度和访问权限。
- **地址计算**：
 - `local_offset`（来自WQE中的 `local_va` 和 MR基址）。
 - `physical_address = mr_base_pa + local_offset + packet_offset`。`packet_offset` 由响应包的序列（First/Middle/Last）决定。

- **权限检查：**确认该MR的权限包含 `LOCAL_WRITE`。检查操作范围（地址+长度）未超出MR边界。
- **失败处理：**若 `l_key` 无效或权限不足，则丢弃Payload，并准备生成携带错误码（如 `LOC_ACCESS_ERR`）的CQE。

阶段四：数据写入（DMA模型）

- **DMA写入引擎：**将报文Payload写入到计算出的 `physical_address` 指向的**内存模型**中。
- **实现注意：**在TLM层面，这通常是一个对 `uvm_mem` 或类似内存模型的 `write()` 调用，而非真实的信号级驱动。应支持模拟分散/聚集（Scatter/Gather）列表，即一个大的读操作可能对应多个非连续的本地内存块。

阶段五：完成通知生成

- **CQE生成器：**仅在处理到 `LAST` 或 `ONLY` 响应包，且所有检查通过、数据写入成功后，才生成CQE。
- **CQE字段填充：**
 - `wr_id`：直接来自原始SQ WQE的 `wr_id`。
 - `status`：成功时为 `WC_SUCCESS`；失败时根据AETH.syndrome或本地错误填写（如 `WC_REM_ACCESS_ERR`, `WC_LOC_LEN_ERR`）。
 - `opcode`：映射为 `IBV_WC_RDMA_READ`。
 - `byte_len`：实际读取的数据总长度。
 - `qp_num`：本地QP号。
- **CQE保序：**同一QP内，CQE的产生和推送顺序必须与SQ WQE的下发顺序严格一致。这要求Agent内部维护一个按 `wr_id` 排序的完成队列。

5.3 UVM组件结构与类定义

Plain Text

```

1 class rdma_requester_rx_agent extends uvm_agent;
2     `uvm_component_utils(rdma_requester_rx_agent)
3
4     // TLM 接口
5     uvm_tlm_analysis_port #(rdma_packet)    in_pkt_port;        // 连接至
网络Monitor
6     uvm_tlm_analysis_port #(rdma_qp_context) in_ctx_port;      // 连接至
QP上下文管理器

```

```

7      uvm_tlm_analysis_port #(rdma_cqe)          cqe_out_port;      // 输出CQE
      至Scoreboard/CQ模型
8
9      // 子组件
10     rdma_requester_rx_driver      driver;
11     rdma_requester_rx_monitor     monitor;
12     rdma_requester_rx_sequencer   sequencer;
13
14     // 配置对象
15     rdma_requester_rx_agent_cfg    cfg;
16
17     // 内部功能模型（可集成在Driver或独立）
18     rdma_psn_manager               psn_mgr;
19     rdma_mr_lookup                 mr_lookup;
20     rdma_dma_write_engine          dma_engine;
21     rdma_cqe_generator             cqe_gen;
22
23     function new(string name, uvm_component parent);
24         super.new(name, parent);
25     endfunction
26
27     virtual function void build_phase(uvm_phase phase);
28         super.build_phase(phase);
29         // 创建端口、配置对象及子组件
30         if(cfg.is_active == UVM_ACTIVE) begin
31             sequencer =
rdma_requester_rx_sequencer::type_id::create("sequencer", this);
32             driver = rdma_requester_rx_driver::type_id::create("driver",
this);
33         end
34         monitor = rdma_requester_rx_monitor::type_id::create("monitor",
this);
35         // 创建功能模型实例
36         psn_mgr = rdma_psn_manager::type_id::create("psn_mgr", this);
37         // ... 创建其他功能模型
38     endfunction
39
40     virtual function void connect_phase(uvm_phase phase);
41         super.connect_phase(phase);
42         if(cfg.is_active == UVM_ACTIVE) begin
43             driver.seq_item_port.connect(sequencer.seq_item_export);
44         end
45         // 连接Monitor的端口到功能模型或Scoreboard
46         monitor.pkt_analysis_port.connect(psn_mgr.analysis_export);
47         // ...
48         cqe_gen.cqe_ap.connect(cqe_out_port);
49     endfunction

```

5.4 关键模型与算法实现要点

5.4.1 PSN管理器 (rdma_psn_manager)

- **状态维护**：为每个QP维护 `expected_psn`。
- **重传逻辑**：当检测到PSN跳跃时，启动一个可配置的定时器。超时后，可产生一个**重传事件**，该事件可被VIP顶层用于触发 `requester_tx_agent` 重发请求包，或直接用于协议重传机制的验证。
- **协议差异**：需支持配置选择**Go-back-N**（RoCEv2典型）或**选择性重传**（iWARP典型）模型。

5.4.2 DMA写入引擎 (rdma_dma_write_engine)

- **TLM级抽象**：核心是调用 `mem_model.write(address, data)`。
- **内存模型接口**：应适配VIP中定义的通用内存模型（如基于 `uvm_mem` 的 `rdma_memory_model`），该模型能模拟物理地址空间、支持MR注册和权限检查。
- **性能建模**：可可选集成延迟模型，模拟DMA写入的延迟时间，用于系统级性能预估。

5.4.3 CQE生成器 (rdma_cqe_generator)

- **顺序队列**：实现一个 `wr_id` 队列，确保即使响应包乱序到达，CQE也按原始WQE顺序产生。
- **错误映射**：需实现从协议错误码（如AETH syndrome中的 `NAK` 代码）到 Verbs 层 `ibv_wc_status` 枚举值的映射表。

5.5 配置与可调试性

`requester_rx_agent` 应提供丰富的配置选项，以平衡仿真速度与验证完备性：

配置参数	类型	默认值	描述
<code>enable_psn_check</code>	bit	1	使能PSN顺序检查。关闭可加速仿真，但失去顺序验证。
<code>enable_mem_protection</code>	bit	1	使能内存密钥（L_Key）和地址范围检查。

<code>retry_mode</code>	<code>enum</code>	<code>RETRY_DISABLED</code>	重传模式：禁用/Go-back-N/选择性重传。
<code>dma_latency_cycles</code>	<code>int</code>	0	模拟DMA写入的延迟周期数。
<code>error_injection_enable</code>	<code>bit</code>	0	使能错误注入，可定向制造PSN错误、权限错误等。
<code>cov_enable</code>	<code>bit</code>	1	使能覆盖率收集，包括PSN跳变、错误类型、操作长度分布等。

Agent应集成全面的**运行时调试信息**输出，并能通过UVM报告机制记录每个处理阶段的结果、遇到的错误以及CQE生成的详细信息，为协议调试提供清晰踪迹。

六、VIP顶层集成与配置

本章将以前述章节定义的核心组件为基础，构建完整的 `rdma_vip_env` 顶层环境。该环境通过一个集中的配置对象 (`rdma_vip_cfg`) 进行驱动，实例化并连接 `requester_tx_agent`、`responder_rx_agent` 和 `requester_rx_agent` 三大功能模块，形成一个自包含、可配置、具备完整数据通路和验证能力的 RDMA 协议事务级模型（TLM）。

6.1 顶层环境框架 (rdma_vip_env)

`rdma_vip_env` 作为整个 VIP 的容器和集成中心，负责所有子组件的生命周期管理、配置分发和端口互联。

Plain Text

```
1  `ifndef RDMA_VIP_ENV_SV
2  `define RDMA_VIP_ENV_SV
3
4  class rdma_vip_env extends uvm_env;
5      `uvm_component_utils(rdma_vip_env)
6
7      // --- 集中式配置对象 ---
8      rdma_vip_cfg    cfg;
9
10     // --- 核心功能代理 ---
11     requester_tx_agent  req_tx_agt;
```

```

12     responder_rx_agent    resp_rx_agt;
13     requester_rx_agent   req_rx_agt;
14
15     // --- 验证组件 ---
16     rdma_scoreboard       scb;
17     rdma_cov_model        cov_model;
18     rdma_assertion_module assert_module;
19
20     // --- TLM 端口（用于跨代理/验证组件通信） ---
21     uvm_tlm_analysis_port #(rdma_transaction) global_trans_ap; // 全局事务广
播
22
23     extern function new(string name = "rdma_vip_env", uvm_component parent
= null);
24     extern virtual function void build_phase(uvm_phase phase);
25     extern virtual function void connect_phase(uvm_phase phase);
26     extern virtual function void report_phase(uvm_phase phase);
27
28 endclass
29
30 `endif // RDMA_VIP_ENV_SV

```

设计要点：

1. **分层集成**：env 作为顶层，不处理具体协议逻辑，仅负责组件的编排、配置和连接，符合 **TLM抽象级别与RDMA协议准确性平衡** 原则中关于高层架构清晰性的要求。
2. **标准化配置接口**：通过 cfg 对象为所有子组件提供统一的参数入口，支持动态重配置，便于构建不同的测试场景。

6.2 集中式配置对象 (rdma_vip_cfg)

配置对象是 VIP 行为控制的“中央枢纽”，它封装了从全局开关到各代理特定策略的所有可调参数，并作为 uvm_config_db 的共享资源。

Plain Text

```

1 class rdma_vip_cfg extends uvm_object;
2     `uvm_object_utils(rdma_vip_cfg)
3
4     // --- 全局与功能开关 ---
5     rand bit    is_active = 1;                // 0: Passive monitor
mode
6     rand bit    enable_psn_check = 1;          // 启用PSN顺序检查
7     rand bit    enable_mem_protection = 1;      // 启用内存保护
(R_Key/L_Key校验)

```

```

8      rand bit      error_injection_enable = 0;          // 全局错误注入开关
9      rand bit      cov_enable = 1;                    // 启用功能覆盖率收集
10     rand int       verbosity_level = UVM_MEDIUM;       // 全局报告详细级别
11
12     // --- 连接与协议参数 ---
13     rand qp_type_e  default_qp_type = RC;              // 默认QP服务类型
14     rand int         default_pmtu      = 1024;          // 默认路径MTU（字节）
15     rand int         default_rnr_retry_cnt = 7;          // 默认RNR重试次数
16
17     // --- 代理级配置 ---
18     requester_tx_config req_tx_cfg;
19     responder_rx_config resp_rx_cfg;
20     requester_rx_config req_rx_cfg;
21
22     // --- 重传与流量控制策略 ---
23     rand retry_mode_e retry_mode = GO_BACK_N;           // 重传模式：DISABLED,
GO_BACK_N, SELECTIVE
24     rand int          ack_delay_cycles;                  // ACK/NACK响应延迟（周期数）
25     rand int          dma_latency_cycles;                 // DMA操作模拟延迟
26
27     // --- 约束 ---
28     constraint valid_cfg_c {
29         default_pmtu inside {256, 512, 1024, 2048, 4096};
30         ack_delay_cycles inside {[0:100]};
31         dma_latency_cycles inside {[1:50]};
32     }
33
34     extern function new(string name = "rdma_vip_cfg");
35 endclass

```

配置对象的作用：

- **统一管理：**避免在各Agent内部散落配置参数，提高可维护性。
- **场景驱动：**测试序列可以通过 `uvm_config_db::set` 在运行时动态修改 `cfg`，快速切换正常、错误、压力等测试场景。
- **参数随机化：**支持对延迟、错误使能等参数进行约束随机化，用于覆盖率驱动验证。

6.3 构建(build_phase)与配置传递

在 `build_phase` 中，环境创建所有子组件实例，并将配置对象传递给它们。

Plain Text

```

1 function void rdma_vip_env::build_phase(uvm_phase phase);
2     super.build_phase(phase);

```

```

3
4 // 1. 获取或创建配置对象
5 if (!uvm_config_db#(rdma_vip_cfg)::get(this, "", "cfg", cfg)) begin
6     cfg = rdma_vip_cfg::type_id::create("cfg");
7     cfg.randomize();
8     `uvm_info("CFG", $sprintf("Created and randomized default
config:\n%s", cfg.sprint()), UVM_LOW)
9 end
10
11 // 2. 设置子组件的配置对象（通过config_db传递上下文）
12 uvm_config_db#(rdma_vip_cfg)::set(this, "req_tx_agt", "cfg", cfg);
13 uvm_config_db#(rdma_vip_cfg)::set(this, "resp_rx_agt", "cfg", cfg);
14 uvm_config_db#(rdma_vip_cfg)::set(this, "req_rx_agt", "cfg", cfg);
15
16 // 3. 创建代理
17 req_tx_agt = requester_tx_agent::type_id::create("req_tx_agt", this);
18 resp_rx_agt = responder_rx_agent::type_id::create("resp_rx_agt",
this);
19 req_rx_agt = requester_rx_agent::type_id::create("req_rx_agt", this);
20
21 // 4. 创建验证组件（根据配置决定）
22 scb = rdma_scoreboard::type_id::create("scb", this);
23 if (cfg.cov_enable) begin
24     cov_model = rdma_cov_model::type_id::create("cov_model", this);
25 end
26 assert_module =
rdma_assertion_module::type_id::create("assert_module", this);
27
28 // 5. 创建全局分析端口
29 global_trans_ap = new("global_trans_ap", this);
30 endfunction

```

6.4 连接(connect_phase)与数据通路集成

connect_phase 是集成工作的核心，负责将各代理的 TLM 端口按照 **RDMA 协议数据流** 连接起来，构成完整的请求-响应环路。这是实现 “**基于事务级（TLM）建模，以快速构建符合协议行为的功能模型**” 目标的关键步骤。

6.4.1 端口连接原理图与映射

以下连接关系实现了原始任务描述的数据流：

1. **TX方向 (Requester):** Verbs层 -> requester_tx_agent -> **网络报文(Request Pkt)**。
2. **RX方向 (Responder):** **网络报文(Request Pkt)** -> responder_rx_agent -> DMA写/Read响应 -> **网络报文(Response Pkt)**。

3. RX方向 (Requester): 网络报文(Response Pkt) -> requester_rx_agent -> DMA写 (仅对Read Response) -> CQE。

Plain Text

```
1 function void rdma_vip_env::connect_phase(uvm_phase phase);
2     super.connect_phase(phase);
3
4     // --- 数据流主线连接 (模拟网络链路) ---
5     // 1. Requester TX 发出的请求包, 送给 Responder RX 和 记分板
6
7     req_tx_agt.driver.req_pkt_ap.connect(resp_rx_agt.monitor.pkt_analysis_export);
8     req_tx_agt.driver.req_pkt_ap.connect(scb.req_pkt_export);
9     req_tx_agt.driver.req_pkt_ap.connect(global_trans_ap); // 广播
10
11     // 2. Responder RX 发出的响应包 (如Read Response, ACK), 送给 Requester RX 和 记分板
12
13     resp_rx_agt.driver.rsp_pkt_ap.connect(req_rx_agt.monitor.pkt_analysis_export);
14     resp_rx_agt.driver.rsp_pkt_ap.connect(scb.rsp_pkt_export);
15     resp_rx_agt.driver.rsp_pkt_ap.connect(global_trans_ap); // 广播
16
17     // --- 旁路通道连接 (上下文与完成通知) ---
18     // 3. 将Responder RX需要的本地RQ WQE上下文传递过去
19     // (假设有一个虚拟的RQ上下文生成器或从外部接口获取)
20     // rq_context_gen.rq_ctx_ap.connect(resp_rx_agt.rq_ctx_export);
21
22     // 4. 完成事件(CQE)收集: 来自Requester RX和Responder RX的CQE送至记分板
23     req_rx_agt.cqe_gen.cqe_ap.connect(scb.cqe_export);
24     resp_rx_agt.cqe_gen.cqe_ap.connect(scb.cqe_export);
25
26     // --- AXI VIP 集成 (作为Responder DMA内存模型) ---
27     // 5. 将Responder RX Agent的DMA引擎的TLM端口连接到Synopsys AXI VIP的从接口。
28     // 这实现了 **AXI VIP作为RDMA VIP responder rx功能基础的设计模式**。
29     //
30     resp_rx_agt.dma_engine.axi_req_port.connect(axi_vip_slave.sequencer.req_export);
31     //
32     axi_vip_slave.memory_response_ap.connect(resp_rx_agt.dma_engine.axi_rsp_export);
33
34     `uvm_info("CONNECT", "AXI VIP slave port connected to responder DMA engine for memory modeling.", UVM_HIGH)
35
36     // --- 验证组件连接 ---
```

```

32 // 6. 全局事务端口连接到覆盖率模型和断言模块（如果使能）
33 if (cfg.cov_enable) begin
34     global_trans_ap.connect(cov_model.trans_export);
35 end
36 global_trans_ap.connect(assert_module.trans_export);
37
38 // 7. 将各Agent Monitor的事务端口也连接到记分板，用于细粒度检查
39 req_tx_agt.monitor.trans_ap.connect(scb.expected_trans_export);
40 resp_rx_agt.monitor.trans_ap.connect(scb.actual_trans_export);
41 req_rx_agt.monitor.trans_ap.connect(scb.actual_trans_export);
42 endfunction

```

6.4.2 关键集成点说明

- **TLM抽象数据流**：所有代理间通过 `rdma_transaction` 类及其衍生的 `rdma_packet` 进行通信，完全在事务级别操作，实现了**快速建模**的目的。
- **AXI VIP 的角色**：`responder_rx_agent` 中的 `dma_engine` 通过 TLM 接口发起“内存写事务”。此接口被连接到 **Synopsys AXI VIP 的 Slave 模型**，后者精确模拟了内存的响应行为（成功、延迟、错误）。这完美体现了“**基于synopsis axi vip基类**”和资料中“**AXI Slave Memory Model for RDMA Responder RX Verification**”的设计模式，用于验证 DMA 写路径的正确性。
- **验证闭环**：`scoreboard` 通过接收请求/响应包和 CQE，可以重构完整的 RDMA 操作序列，验证端到端的协议正确性和数据一致性。

6.5 运行控制与脚本化用例示例

顶层环境通过标准的 UVM Phase 机制运行。用户通过在测试用例 (`uvm_test`) 中配置环境并发送虚拟序列来控制 VIP 的行为。

Plain Text

```

1 class simple_rdma_write_test extends uvm_test;
2     `uvm_component_utils(simple_rdma_write_test)
3     rdma_vip_env env;
4     virtual_rdma_sequence vseq;
5
6     function void build_phase(uvm_phase phase);
7         super.build_phase(phase);
8         // 创建并配置环境
9         env = rdma_vip_env::type_id::create("env", this);
10        // 可在此处通过config_db覆盖默认配置，例如关闭PSN检查进行错误测试
11        // uvm_config_db#(int)::set(this, "env.cfg", "enable_psn_check",
12        0);
13    endfunction

```

```

13
14     task run_phase(uvm_phase phase);
15         phase.raise_objection(this);
16         vseq = virtual_rdma_sequence::type_id::create("vseq");
17         vseq.start(null); // 启动虚拟序列，协调多个Agent的行为
18         phase.drop_objection(this);
19     endtask
20 endclass
21
22 // 一个简单的虚拟序列示例
23 class virtual_rdma_sequence extends uvm_sequence;
24     `uvm_object_utils(virtual_rdma_sequence)
25     task body();
26         // 1. 初始化连接 (模拟ibv_modify_qp)
27         `uvm_do_on(init_qp_seq, p_sequencer.req_tx_seqr)
28         // 2. 并发执行多个RDMA操作
29         fork
30             begin `uvm_do_on_with(rdma_write_seq, p_sequencer.req_tx_seqr,
31 { length == cfg.default_pmtu; }) end
32             begin `uvm_do_on(rdma_read_seq, p_sequencer.req_tx_seqr) end
33         join
34         // 3. 等待所有操作完成 (通过CQE Polling模拟)
35         #1000ns;
36     endtask
37 endclass

```

6.6 验证组件集成总结

至此，一个完整的、基于 UVM-TLM 的 RDMA VIP 顶层框架已集成完毕：

- **协议准确性**：通过 `rdma_transaction` 的精确约束、各 Agent 内部的协议状态机（如 PSN 管理、WQE 处理）以及与 **Synopsys AXI VIP** 的精确内存模型集成来保证。
- **可配置性**：通过 `rdma_vip_cfg` 对象，用户可以灵活控制协议检查的严格度、错误注入、性能参数等，以平衡仿真速度与验证深度。
- **可观测性**：通过全局和局部的 TLM 分析端口、集成的 `scoreboard`、`cov_model` 和 `assert_module`，提供了从事务流到信号级的多层次调试和验证覆盖能力。
- **可重用性**：整个 `rdma_vip_env` 可以作为黑盒 VIP 集成到更大的 SoC 验证环境中，或作为独立单元用于驱动开发与架构探索。

此框架实现了从 Verbs 接口命令到网络报文收发，再到最终内存访问和完成通知的完整 RDMA 操作闭环建模，为早期架构验证和协议合规性测试提供了坚实的高性能基础。